

PROJECT

Evaluation

- 80% Project+20% Lab
- Most of Lab notices are about the project.
 - Better lab results -> better project result
- Project evaluation: Report + Github + Oral Presentation
- Last Lab session for presentation
- Team with maximum 2 members (If you do not find a partner, do it alone)

Project topic: Building a knowledge supported AI assistant

- **Bridging the Gap Between the Semantic Web and Generative AI**
- **The Goal:** Transform messy, unstructured web data into a high-precision **Knowledge Graph (KG)** to power a **Retrieval-Augmented Generation (RAG)** assistant.
- **The Value:** You won't just build a chatbot; you will build a **system of record** that knows *why* it gives the answers it does.

The journey

- **Extract:** Turn raw HTML into cleaned text and identified entities (NLP).
- **Structure:** Formalize facts using **RDF** and domain **Ontologies** (Semantic Web).
- **Enhance:** Train **Knowledge Embeddings** for similarity and link prediction.
- **Solve:** Build a **RAG** system that uses structured facts to "ground" LLM responses, reducing hallucinations.



ESILV

ENGINEERING SCHOOL
DE VINCI PARIS

WEB MINING: FACING THE REALITY OF RAW WEB DATA

Start by collecting web data



DE VINCI

RESEARCH
CENTER

Why This Course?

- Modern AI systems consume massive amounts of web data
- LLMs, search engines, recommender systems all rely on web content
- However:
 - Web data is not designed for machines
 - Knowledge is rarely explicit
- **Core question of the course:**
 - *How do we turn raw web data into usable knowledge for AI systems?*

RoadMap

End-to-end perspective

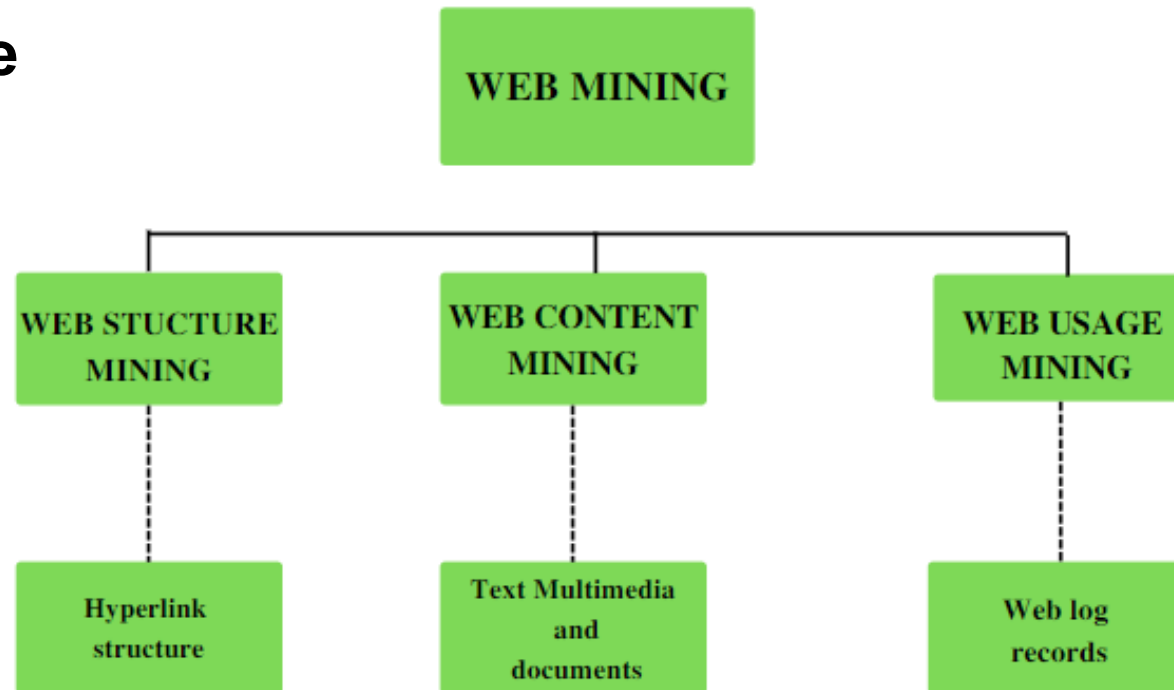
1. Collect web data
2. Extract information
3. Structure knowledge
4. Formalize semantics
5. Learn representations
6. Build intelligent systems

This session: Step 1 — Data collection and its limits

What Is Web Mining?

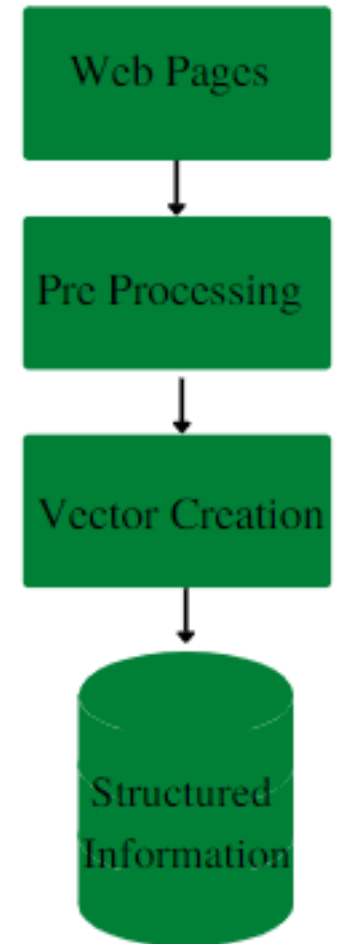


- Extract useful knowledge from web data
- It targets three main data types:
 - **Web Content**
 - **Web Structure**
 - **Web Usage**



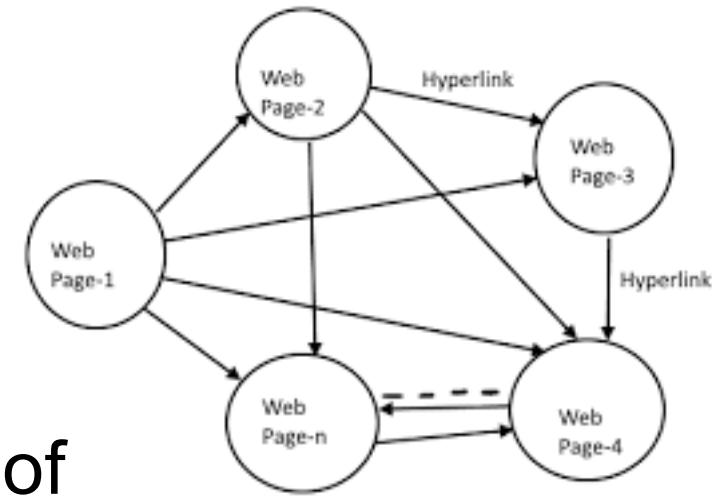
Web Content Mining

- Extracting meaningful information from web pages.
 - Text, Image, Audio, Video
- Difficulties: Web data are generally **semi-structured and/or unstructured**. We need to convert them into structured data. Mostly **Text Mining** techniques (we will learn some solutions later)
- Examples:
 - Classifying the web documents into categories.
 - Identify topics of web documents.
 - Finding similar web pages across the different web servers.
 - Applications related to relevance.



Web Structure Mining

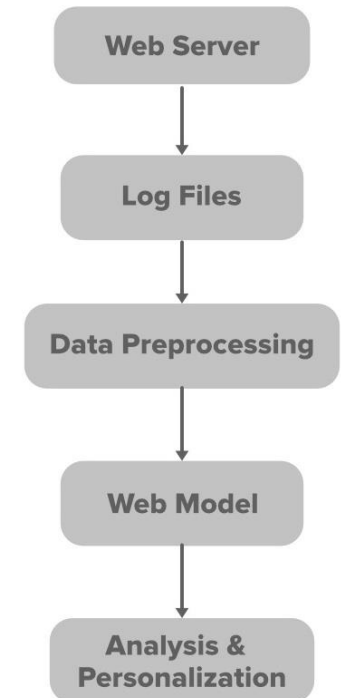
- Discovering structure information from the web. Usually, the structure of the web graph consists of web pages as **nodes**, and hyperlinks as **edges** connecting related pages.
- Mostly **Graph Mining** techniques (You will learn on the 5th year)
- Examples:
 - Information retrieval in social networks (graph)
 - To find out the relevance of each web page.
 - Used in Search engines to find the relevant information.



Web Usage Mining

- Identifying or discovering interesting **usage patterns** from large data sets.
- These patterns may help understanding the behaviours or **profile** of the users.
- Usually, user access data is in form of logs (structured data).
- Applications:
 - Privatization of web content (recommendation system)
 - Pre-recovery (to improve the performance of we servers)

Flow of web Personalization



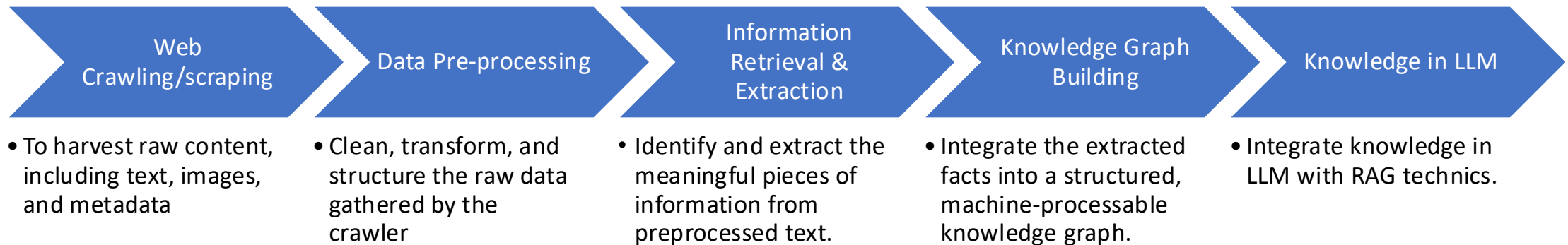
Challenges in web mining

- Deluge of data
 - Large-scale data with unknown distributions
 - Often little ratio of data is available and/or useful
 - Sampling is hard
 - Data is noisy
 - Data is not always trustworthy
 - Heterogeneous data and sources
- Diversity of user profiles
 - Paradox of data availability and privacy for personalization purposes
 - Heterogeneity of connections
- Evaluation Dilemma
 - Building ground truth is costly

What will we learn in this course?

- Web structure mining? No
 - Mostly graph mining techniques, you will have a such module in A5
- Web usage mining? No
 - Mostly applications on database management and recommendation system building.
- Web content mining? Yes
 - We will learn how to build knowledge base from flat text.

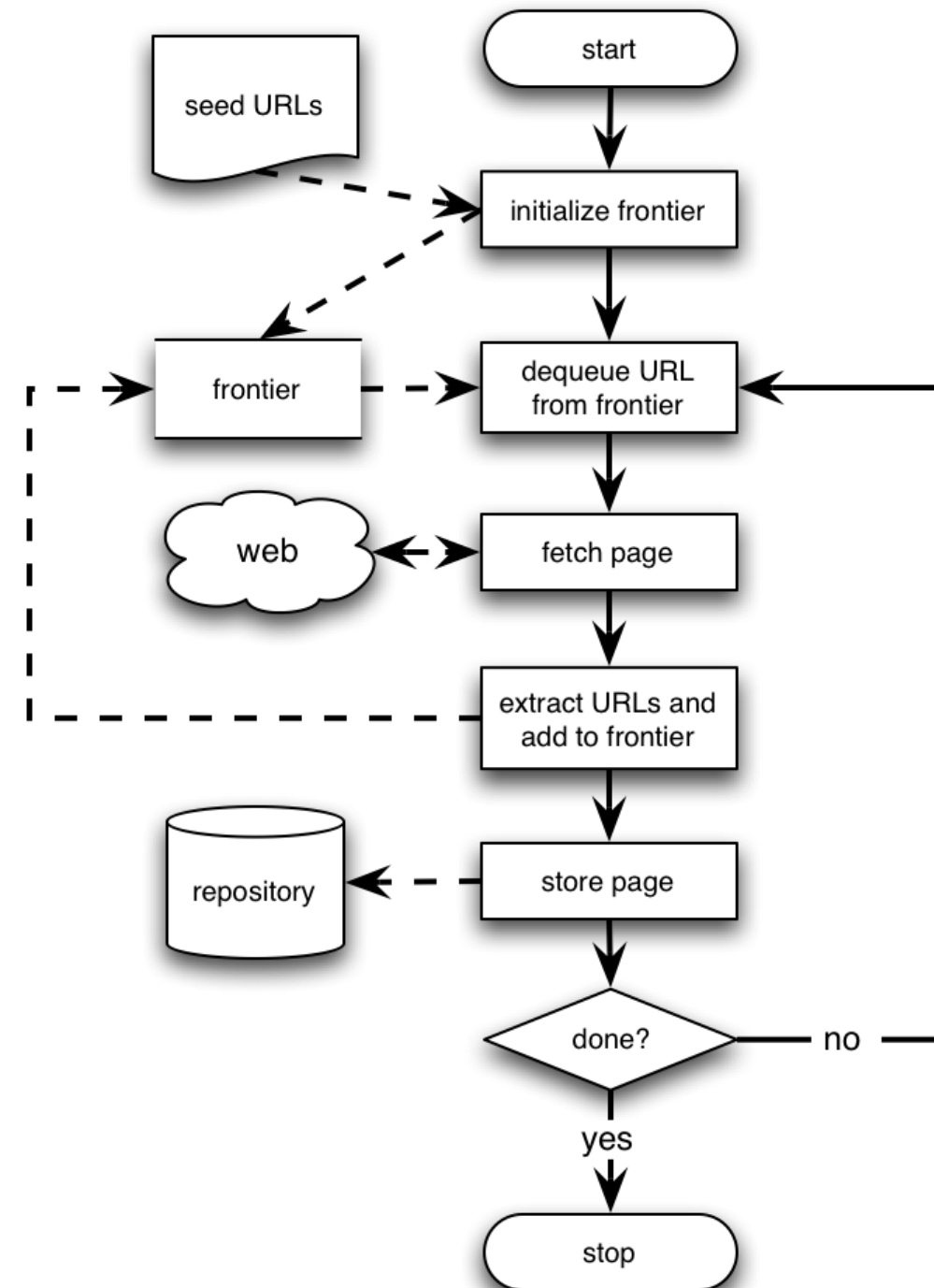
The Overall Pipeline



Web crawling/ scrapping

How Web Crawlers Work

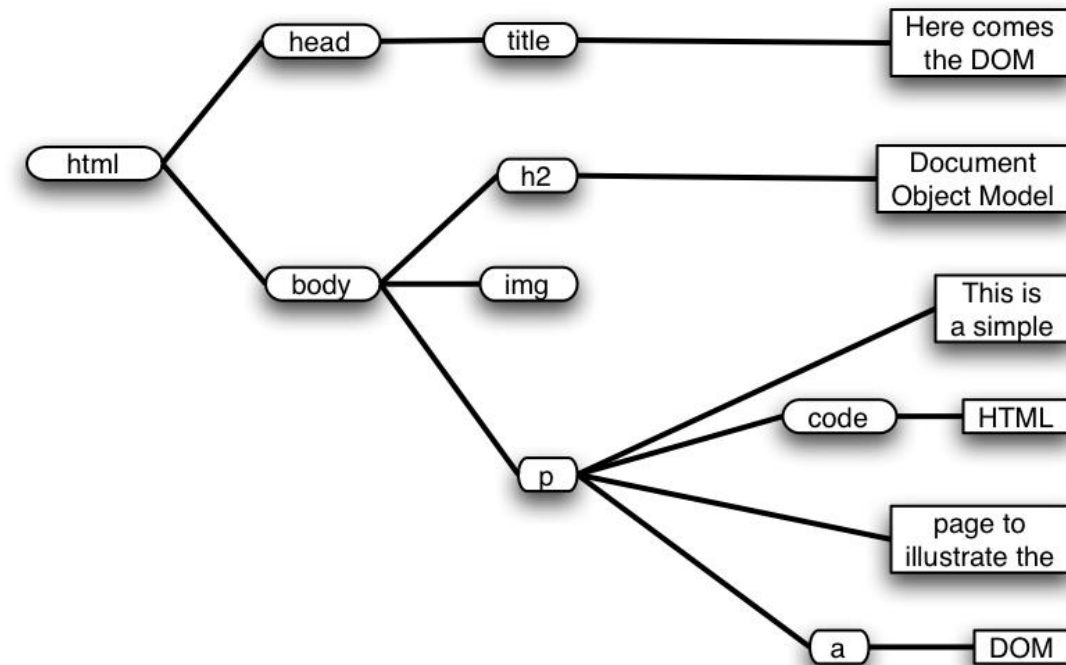
- This is a sequential crawler
- Seeds can be any list of starting URLs
- Fetching Pages: The crawler downloads web pages by sending HTTP requests.
- Order of page visits is determined by frontier data structure
- Stop criterion can be anything



How Web Crawlers Work

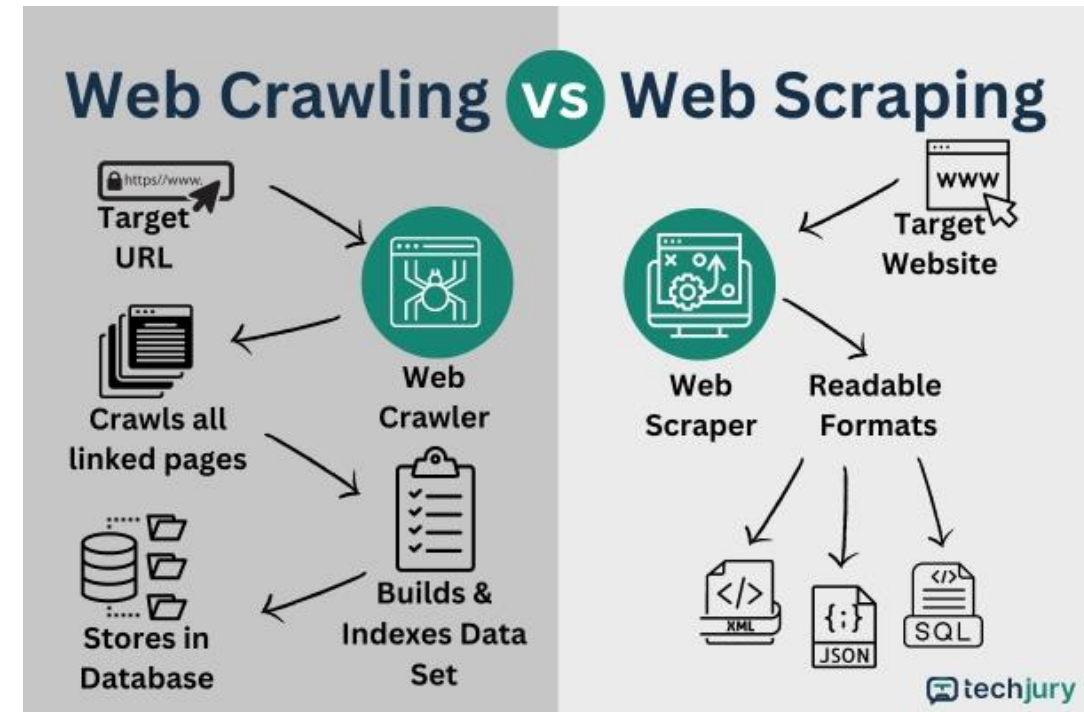
- Parsing & Link Extraction:
 - Use HTML parsers (e.g., **BeautifulSoup**) to extract all the links (anchor tags) from the page.
 - Filter out irrelevant or duplicate URLs.
- Queue Management:
 - Discovered URLs are added to a frontier (queue) to be crawled later.
 - Scheduling algorithms determine the order of crawling.

```
<html>
  <head>
    <title>Here comes the DOM</title>
  </head>
  <body>
    <h2>Document Object Model</h2>
    
    <p>
      This is a simple
      <code>HTML</code>
      page to illustrate the
      <a href="http://www.w3.org/DOM/">DOM</a>
    </p>
  </body>
</html>
```



Web scraping

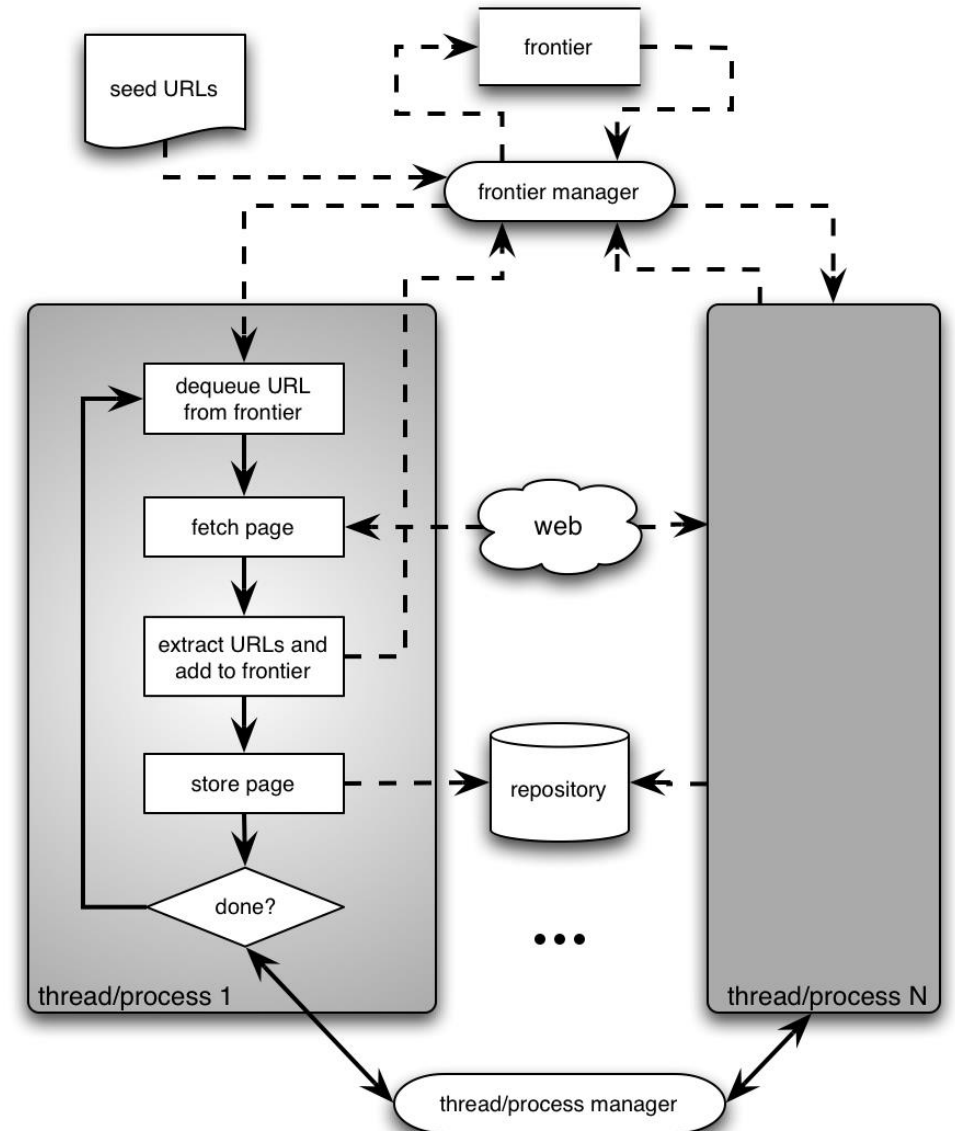
- Web scraping is the process of using bots to extract **specific data** from **a target website**. Web scraping uses web crawlers to scan and store all the content from a webpage (or a set of webpages) and then uses a web scraping tool or script to extract specific data from that content.



Advanced Crawling & Data Collection



- **Handling Dynamic Content:** Modern websites use AJAX/JavaScript
 - **Solution:** Employ headless browsers (e.g., Puppeteer, Selenium) to render pages
- **Distributed Crawling Architectures:** Utilize multiple agents to cover the web efficiently
 - Address challenges such as load balancing, duplicate removal, and error recovery
- **Dealing with Duplicates:**
 - Use content hashing (e.g., MD5, SHA-1) to detect and remove duplicate pages.



More issues on crawling & data collection

- Politeness, Ethical & Legal Considerations:
 - Crawlers must respect the guidelines specified in a website's **robots.txt** file.
 - Implement delays between requests to avoid overloading a server. (you are not conducting a DDoS attack!)
 - Avoid collecting personal data without permission, and ensure that the crawling activity complies with legal requirements.

A robots.txt file tells search engine crawlers which URLs the crawler can access on your site.

[Here](#) for more info on robots.txt

```
User-agent: *  
Disallow: /cgi-bin/  
Disallow: /tmp/  
Disallow: /junk/
```

- The best way of learning web crawling is implementing by yourselves.

Some Python packages for web crawling/scraping

- [Requests](#): making http request
- [BeautifulSoup](#): parsing the html content
- [Selenium](#): automating web browser (with user defined header)
- [Lxml](#) module: Parsing XML and HTML with high performance. (an XML used for storage can be very large)
- [Urllib](#): working with URLs
- [PyautoGUI](#): Controlling the mouse and keyboard to automate tasks
- [Schedule](#): scheduling python functions to run for periodical or interval missions
- More on <https://www.geeksforgeeks.org/python-web-scraping-tutorial/>

One utilisation: html parsing

- **The Bridge from Code to Content** (Example: the source of page: https://en.wikipedia.org/wiki/Semantic_Web)
- **Definition:** Parsing is the process of analyzing a string of HTML code and converting it into a structured **Tree Map** (the Document Object Model or DOM).
- **The Problem:** Raw HTML is designed for browsers, not for AI. It contains "noise" like scripts, CSS, and layout tags.
- **The Goal:** To isolate the **signal** (data/text) from the **noise** (formatting).

BeautifulSoup - The Surgical Tool for Precision

- A Python library for pulling data out of HTML and XML files.
- It creates a parse tree that allows you to "query" specific tags using CSS selectors or tag names.
- **Best Use Case:** When you know the exact structure of a page.
 - *Example:* "Find every <h3> tag inside the <div id='results'>."
- **Strength:** Total control. You can extract specific table cells or hidden metadata precisely.

An intelligent refiner: Trafilatura

- A specialized tool designed for Web Discovery and Main Text Extraction.
- How it works: It uses advanced heuristics to automatically identify the "Main Article" and discard sidebars, ads, and footers.
- Best Use Case: When crawling hundreds of different websites where you cannot write custom rules for each one.
- Strength: "Battery-included" boilerplate removal. It returns clean Markdown, making it perfect for LLMs and RAG pipelines.

Which to choose?

Feature	BeautifulSoup	Trafilatura
Logic	Manual / Rule-based	Automatic / Heuristic
Effort	High (must inspect HTML)	Low (one function call)
Best For	Structured data (Prices, Lists)	Unstructured data (Articles, Blogs)
Cleanliness	You must clean it yourself	Returns ready-to-use text/Markdown

Use **BeautifulSoup** for specific, known targets;
use **Trafilatura** for broad, content-heavy knowledge gathering.

See demo: `compare_soup_trafilatura.py`

Tools and frameworks

- Scrapy: An open-source framework in Python for large-scale crawling and web scraping.
- Apache Nutch: A highly extensible and scalable open-source web crawler built on **Hadoop**. (In Java)
- Heritrix: The Internet Archive's crawler, designed for archival purposes.

Live Demonstration: Collecting web data

- See demo: `webCrawler.py`
- Some problems:
 - Navigation menus included in text
 - Repeated sections
 - No explicit entities
 - No explicit relations
 - Everything is just flat text

What Do We Actually Get?

- Large amount of text
- Weak signal-to-noise ratio
- No explicit entities
- No explicit relations

We collect only text, not knowledge!

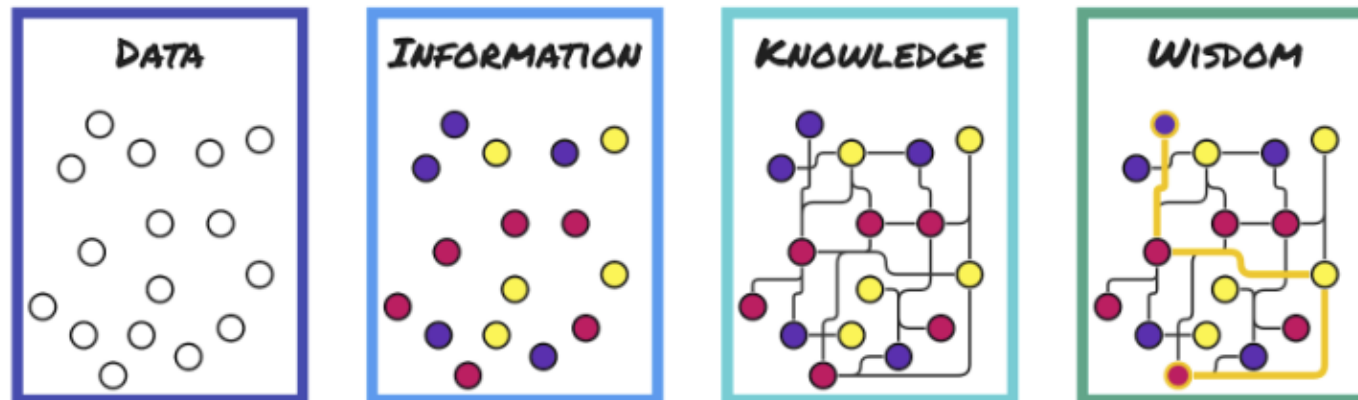
Raw text is a problem

- Core limitations
 - Ambiguity
 - Synonymy
 - No identity resolution
 - No formal structure
 - No inference capability
- Example: “I work at ESILV in Paris”
 - “Paris” ≠ unique entity
 - “works at” ≠ explicit relation

Text vs Knowledge

Raw text	Knowledge
Unstructured	Structured
Implicit meaning	Explicit meaning
Human-readable	Machine-interpretable
Context-dependent	Context-independent

Text may contain knowledge, but it is not knowledge.



How to represent knowledge

- Objective: encoding information about the world into formats that **informatic/AI systems** can utilize to solve complex tasks.
- Core Methods of Knowledge Representation (we'll see this in next sessions)
 - Logic-Based Systems
 - Structured Representations
 - Probabilistic Models
 - Distributed Representations
- Knowledge graph triplet: (*subject, predicate, object*)

Knowledge Graph triplet

(subject, predicate, object)

- Each triplet expresses a **fact** about the world in a way that is explicit and machine-interpretable.
 - Subject: the entity being described
 - Predicate: the relation or property
 - Object: the value or another entity related to the subject

Examples

- Example 1: Factual Relation

- “Paris is the capital of France.” -> (Paris, capitalOf, France)
 - Subject: Paris (entity)
 - Predicate: capitalOf (relation)
 - Object: France (entity)

- Example 2 — Attribute (Literal Value)

- “Paris has a population of 2.1 million.” -> (Paris, population, 2100000)
 - Object is a **literal**, not an entity

- Example 3 — Person and Organization

- “Tim Berners-Lee worked at CERN.” -> (Tim_Berners-Lee, workedAt, CERN)

Graph interpretation

- Triplets naturally form a **graph**:
 - **Nodes**: subjects and objects (entities or values)
 - **Edges**: predicates (relations)
 - Example: Tim Berners-Lee — workedAt —> CERN
 - A collection of triplets forms a **knowledge graph**.
- Multiple triplets can describe the same entity:
 - (Paris, capitalOf, France)
 - (Paris, locatedIn, Europe)
 - (Paris, population, 2100000)

Advantages of triplets

- **Explicit Semantics**
 - Relations are named and explicit
 - Meaning is not inferred from text context
- **Machine Interpretability**
 - Triplets can be: Queried, Merged across sources, and Reasoned over.
- **Triplets vs Natural Language:** (We'll see these advantages later)

Natural Language	Knowledge Triplet
Ambiguous	Explicit
Context-dependent	Context-independent
Hard to query	Easy to query
Human-oriented	Machine-oriented

Relation to Later Topics

- **RDF** formulates ontology
 - RDF (Turtle syntax):

```
:Paris :capitalOf :France .  
:Paris :population 2100000 .
```
- **SPARQL** queries match patterns of triplets
- **Knowledge graph embeddings** learn vector representations of triplets
- **RAG systems** retrieve knowledge derived from triplets

Next session

- How to convert plain text into Triplet?